A. Core Game Server

Core Game Server는 게임의 **절대 권위(authoritative)**를 가지는 서버로, 모든 전투 결과와 상태를 최종 결정한다.

역할

- 게임 방 생성 및 관리
- 플레이어 매칭 및 입장 처리
- 턴 진행 및 순서 제어
- 전투 계산 (데미지, 힐, 상태 이상 등)
- 승패 판단 (전멸 조건)
- 게임 상태 스냅샷 생성 및 전송

특징

- 클라이언트는 **결과를 계산하지 않는다**
- 모든 계산은 서버에서 수행 후 결과만 전달
- 치트 방지의 핵심 계층

데이터 흐름

```
Client Input (공격/스킬)
↓
Game Server
↓
전투 계산
↓
State Update Broadcast
```

향후 확장

- AI / NPC 참여
- 리플레이 시스템
- 상태 스냅샷 압축 / 델타 전송

B. Realtime Layer - RootNet

RootNet은 Unity/WebGL 환경을 염두에 둔 **클라이언트 중심 네트워크 라이브러리** 초안이다.
Mirror 의존 구조를 제거하고, Transport / Protocol / Unity Adapter를 분리한 형태로 다시 설계하는 것이 목표다.

현재 방향은 다음과 같다.

- **클라이언트 전용** 구조
- **Host 모델 미지원**
- **WebSocket 기반**
- 전송 프레임은 **binary frame 기준**
- 시스템 메시지는 **JSON payload**
- 실시간 메시지는 **Binary payload**
- Unity와 네트워크 코어를 분리
- `static` 전역 네트워크 상태 제거
- 플랫폼 차이는 **composition backend**로 분리

1. 현재 아키텍처 개요

전체 흐름은 다음과 같다.

```
Game / Unity Layer
   ↓
RootNetClientBehaviour
   ↓
NetClient
   ↓
IClientTransport
   ↓
WebSocketTransport
   ↓
IWebSocketClientBackend
   ├─ NativeWebSocketBackend
   └─ WebGLWebSocketBackend
```

핵심 원칙:

1. **Transport는 바이트만 다룬다**
2. **Protocol이 JSON/Binary를 구분한다**
3. **Unity MonoBehaviour는 어댑터 계층에만 둔다**
4. **플랫폼별 구현은 partial이 아니라 composition으로 분리한다**

2. 패킷 포맷

RootNet은 WebSocket 위에서 **binary frame**만 사용한다.

패킷 헤더는 현재 다음과 같다.

```
[1 byte  : format]
[2 bytes : messageId]
[payload]
```

format

- `1` = JSON 시스템 메시지
- `2` = Binary 실시간 메시지

예시

시스템 메시지

- HelloRequest
- HelloResponse
- PingMessage
- PongMessage

실시간 메시지

- MoveInputMessage
- AttackInputMessage
- 이후 Snapshot / State / Skill / Combat Event 등 확장 예정

3. JSON / Binary 분리 이유

시스템 메시지: JSON

시스템 메시지는 다음 목적을 가진다.

- Python 서버 임시 테스트
- Java 서버와의 빠른 연동
- 디버깅 편의성
- 프로토콜 가시성 확보

현재 Unity 쪽에서는 `JsonUtility`를 사용한다.

주의:

- `[Serializable]` class 기반
- `public field` 사용
- property 기반 직렬화는 사용하지 않음

실시간 메시지: Binary

실시간 메시지는 다음을 위해 binary를 사용한다.

- 패킷 크기 절감
- 직렬화 비용 절감

- GC 최소화
- 이동 / 공격 / 입력 / 스냅샷 처리 최적화

4. 폴더 구조

현재 폴더 구조는 다음과 같다.

```
Assets/
└─ RootNet/
   └─ Runtime/
      ├─ Abstractions/
      │  ├─ IClientTransport.cs
      │  ├─ IMessageSerializer.cs
      │  ├─ ISystemMessageSerializer.cs
      │  ├─ IBinaryMessageCodec.cs
      │  ├─ ISystemMessage.cs
      │  ├─ IBinaryMessage.cs
      │  ├─ INetLogger.cs
      │  └─ IWebSocketClientBackend.cs
      │
      ├─ Core/
      │  ├─ ClientConnectionState.cs
      │  ├─ MessageFormat.cs
      │  ├─ NetClientConfig.cs
      │  ├─ NetClient.cs
      │  └─ NetClientBehaviour.cs
      │
      ├─ Protocol/
      │  ├─ CompositeMessageSerializer.cs
      │  ├─ JsonSystemMessageSerializer.cs
      │  ├─ SystemMessageRegistry.cs
      │  ├─ BinaryMessageRegistry.cs
      │  ├─ NetWriter.cs
      │  └─ NetReader.cs
      │
      ├─ Messages/
      │  ├─ System/
      │  │  ├─ HelloRequest.cs
      │  │  ├─ HelloResponse.cs
      │  │  ├─ PingMessage.cs
      │  │  └─ PongMessage.cs
      │  │
      │  └─ Realtime/
      │     ├─ MoveInputMessage.cs
      │     ├─ AttackInputMessage.cs
      │     └─ Codecs/
      │        ├─ MoveInputMessageCodec.cs
      │        └─ AttackInputMessageCodec.cs
      │
      ├─ Logging/
      │  ├─ NullNetLogger.cs
      │  └─ UnityNetLogger.cs
```

```
        │
        ├─ Transports/
        │  └─ WebSocket/
        │     ├─ WebSocketTransport.cs
        │     ├─ NativeWebSocketBackend.cs
        │     ├─ WebGLWebSocketBackend.cs
        │     ├─ UnsupportedWebSocketBackend.cs
        │     └─ WebSocketBackendFactory.cs
        │
        ├─ Unity/
        │  ├─ RootNetClientBehaviour.cs
        │  └─ Debug/
        │     ├─ RootNetDebugInputBehaviour.cs
        │     └─ RootNetDebugLogBehaviour.cs
        │
        └─ Bootstrap/
           └─ RootNetBootstrap.cs
```

5. 각 계층 역할

Abstractions

코어 인터페이스 모음.

- Transport 인터페이스
- Serializer 인터페이스
- Message marker 인터페이스
- Logger 인터페이스
- WebSocket backend 인터페이스

Core

네트워크 클라이언트의 중심 계층.

- 연결 상태
- timeout
- transport 이벤트 바인딩
- 메시지 핸들러 등록
- send / receive 흐름 제어

Protocol

메시지 직렬화 / 역직렬화 담당.

- JSON 시스템 메시지 처리
- Binary codec 처리
- header wrap / unwrap
- registry 관리

Messages

실제 송수신 메시지 타입 정의.

Transports

전송 구현 계층.

- 공통 WebSocketTransport
- Native backend
- WebGL backend
- Unsupported backend
- backend factory

Unity

Unity 생명주기 어댑터 계층.

- MonoBehaviour 연결점
- Update / OnDestroy 연결
- 디버그 컴포넌트 분리

Bootstrap

기본 registry, codec, serializer, transport 생성 로직 담당.

6. 현재 핵심 클래스 설명

NetClient

실제 클라이언트 네트워크 세션 객체.

역할:

- connect / disconnect
- timeout 관리
- transport 이벤트 처리
- 메시지 handler 등록
- 시스템/바이너리 메시지 전송

WebSocketTransport

IClientTransport 구현체.

역할:

- backend 위에 얇은 transport 계층 제공
- send queue 관리
- main-thread event queue 관리
- backend 이벤트를 transport 이벤트로 변환

IWebSocketClientBackend

플랫폼별 실제 WebSocket 구현 인터페이스.

구현체:

- `NativeWebSocketBackend`
- `WebGLWebSocketBackend`
- `UnsupportedWebSocketBackend`

RootNetClientBehaviour

Unity에서 RootNet을 붙이는 공식 진입점.

역할:

- NetClient 생성/소유
- Update에서 `EarlyUpdate`, `LateUpdate` 호출
- Connect/Disconnect 래핑
- Unity 오브젝트 수명과 네트워크 수명 연결

7. 현재 설계 포인트

7.1 partial 대신 composition

초기에는 `partial`로 플랫폼 분리하려 했으나, C# partial method 제약과 구조적 가독성 문제 때문에 **backend composition** 방식으로 전환했다.

이 방식의 장점:

- 플랫폼 차이를 클래스 외부로 분리
- 테스트 backend 주입 가능
- fake backend 작성 쉬움
- WebSocketTransport가 플랫폼 세부 구현을 몰라도 됨

7.2 Connect/Disconnect는 async

Transport의 `ConnectAsync`, `DisconnectAsync`는 비동기로 두고, `Send`는 빈도가 높으므로 동기 enqueue 방식으로 유지한다.

정리:

- `ConnectAsync` / `DisconnectAsync` → async
- `Send` → immediate enqueue
- 실제 송신은 `LateUpdate`에서 flush

7.3 Unity Layer 분리

샘플용 MonoBehaviour 하나에 모든 책임을 몰지 않고:

- 핵심 진입점
- 디버그 입력
- 디버그 로그

를 분리하는 방향으로 정리한다.

8. 현 시점 상태

현재는 **아키텍처와 코어 구조 정리 단계**다.

정리된 것:

- transport abstraction
- protocol layer 분리
- JSON/Binary 이중 포맷
- composition backend 구조
- Unity adapter 분리 방향
- bootstrap 구조

아직 구현해야 할 것:

- `NativeWebSocketBackend` 실제 구현
- `WebGLWebSocketBackend` 실제 구현
- Python 테스트 서버
- HelloRequest / HelloResponse roundtrip
- MoveInputMessage binary roundtrip

9. 다음 우선순위

1순위

`NativeWebSocketBackend` 실제 구현

예상 포함 작업:

- `ClientWebSocket`
- async connect
- receive loop
- thread-safe receive queue
- binary send
- close 처리

2순위

Python WebSocket 테스트 서버 작성

최소 목표:

- HelloRequest 수신
- HelloResponse 응답
- Ping/Pong 테스트

3순위

WebGL backend bridge 구현

포함 작업:

- `.jslib`
- JS WebSocket bridge
- binaryType = arraybuffer
- JS -> C# queue pump

10. 장기 확장 후보

현재 설계 위에 이후 붙일 수 있는 것들:

- reconnect policy
- heartbeat / ping scheduler
- auth/session state machine
- room / lobby layer
- snapshot replication
- delta compression
- unreliable channel (WebRTC 등)
- prediction / reconciliation
- encryption / secure session
- packet metrics / profiler

12. 메모

- 시스템 메시지는 JSON payload지만, 전송 프레임은 binary frame 기준이다.
- JSON은 "성능용"이 아니라 "서버 인터페이스/디버깅용"이다.
- 실시간 패킷은 반드시 binary로 유지한다.
- MonoBehaviour는 코어가 아니라 Unity adapter 계층에만 둔다.
- 앞으로 `RootNetClientBehaviour`는 샘플이 아니라 핵심 엔트리 포인트가 될 수 있다.

13. 현재 한 줄 요약

RootNet은 **Unity/WebGL 대응을 고려한 클라이언트 중심 WebSocket 네트워크 라이브러리 초안**이며, **Transport / Protocol / Backend / Unity Adapter를 분리한 구조**로 다시 설계되고 있다.

C. S2E Engine (Support-to-Earn)

S2E Engine은 관전자가 게임에 개입하는 **핵심 수익 및 영향 시스템**이다.

역할

- 후원 아이템 사용 처리 (공격 / 힐)
- 서포터 장비 기반 효과 증폭 계산
- 실시간 전투 영향 적용
- 이벤트를 Core Game Server로 전달

핵심 요구사항

- 입력 → 적용까지 **0.5초 이내**
- 게임 결과에 직접적인 영향

처리 흐름

```
Spectator Action (아이템 사용)
↓
S2E Engine
↓
효과 계산 (증폭 포함)
↓
Game Server 전달
↓
즉시 전투 반영
```

특징

- 단순 결제가 아니라 **게임 입력으로 처리됨**
- "돈 → 이벤트 → 전투 영향" 구조

위험 요소

- latency 증가 시 게임 체감 붕괴
- 동시 입력 폭주 처리 필요

D. Payment / Ledger System

Payment / Ledger는 모든 금전 흐름을 처리하는 **정산 및 회계 시스템**이다.

역할

- PG 결제 처리 (카드 / 휴대폰 등)
- 결제 성공 이벤트 생성
- 매출 집계 (Total Sales)
- 정산 로직 수행
- 유저별 수익 배분 기록
- 원장(Ledger) 관리

정산 흐름

```
Total Sales
↓
(1) 세금 + PG 수수료 차감
↓
Net Pot
↓
(2) 소각 / 바이백 / 위로금 차감
↓
Final Pool
↓
(3) 승리 팀 분배 (1:9)
```

특징

- 게임 서버와 **분리된 독립 시스템**
- 금융/회계 성격 → 안정성 최우선

중요 포인트

- 모든 계산은 **서버 단일 기준**
- 클라이언트 계산 금지
- 로그/추적 가능 구조 필수

E. Web3 Bridge

Web3 Bridge는 외부 블록체인 시스템과 게임을 연결하는 계층이다.

역할

- 지갑 연결 및 NFT 보유 여부 확인
- 플레이어 자격 검증 (RWA NFT 필수)
- 스테이킹 연동 처리
- NFT 변환 API 제공

주요 기능

1. NFT 보유 검증

```
Client → Wallet Scan → Web3 Bridge → 결과 반환
```

- NFT 없으면 spectator 제한

2. 스테이킹 연동

```
External Site (펑크비즘)
↓
Game Server API 호출
↓
유저 인벤토리 지급
```

3. NFT 변환 API

- 조회(Scan): 게임 아이템 목록 반환
- 소각(Burn): 게임 아이템 삭제 또는 잠금

특징

- 민팅은 외부 시스템에서 수행
- 게임 서버는 **조회 + 삭제만 담당**

F. Admin System

Admin 시스템은 운영 및 밸런스 관리를 위한 내부 도구이다.

역할

- NFT 아바타 인증 승인/거절
- 게임 밸런스 수정 (엑셀 업로드)
- 매출 및 결제 로그 확인
- 유저 상태 관리

주요 기능

1. 아바타 심사

- NFT 이미지 검증
- 승인 / 거절 처리

2. 밸런스 관리

- 700종 카드 데이터 수정
- 엑셀 기반 일괄 업로드

3. 매출 분석

- 가격대별 판매 통계
- 유저별 결제 내역 조회

특징

- 게임 클라이언트와 완전히 분리
- 내부 운영 전용

전체 요약

```
RootNet은 Core Game Server, S2E Engine, Payment/Ledger, Web3 Bridge, Admin System
으로 구성된 전체 플랫폼 중,
Realtime Layer를 담당하는 클라이언트-서버 통신 핵심 계층이다.
```